Bazele inteligenței artificiale

Laborator – Grigore Ionuț

Laboratory contents

1. Introduction to Python
2. Linear models
3. Nonparametric models
4. Decision trees. Ensemble methods
5. Dimensionality reduction
6. Clustering
7. Test 1
8. Introduction to PyTorch → **today's lab**
9. Multilayer perceptrons
10. Convolutional neural networks
11. Recurrent neural networks
12. Attention mechanisms
13. Test 2
14. Recoveries

Content

- Data manipulation in PyTorch
- Linear algebra in PyTorch
- Automatic differentiation in PyTorch

1) Data manipulation in PyTorch

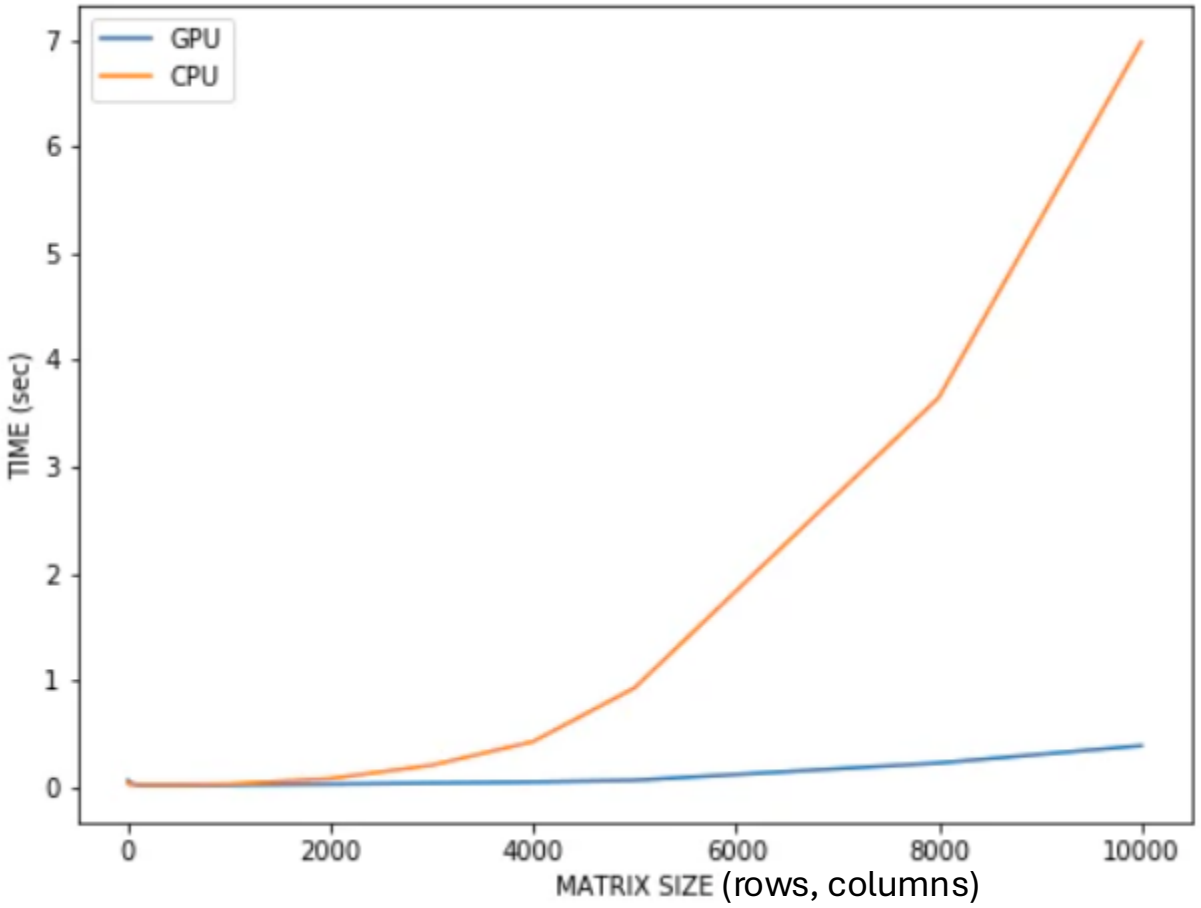
first lab → **NumPy + numpy array**

this lab → **PyTorch + tensors**

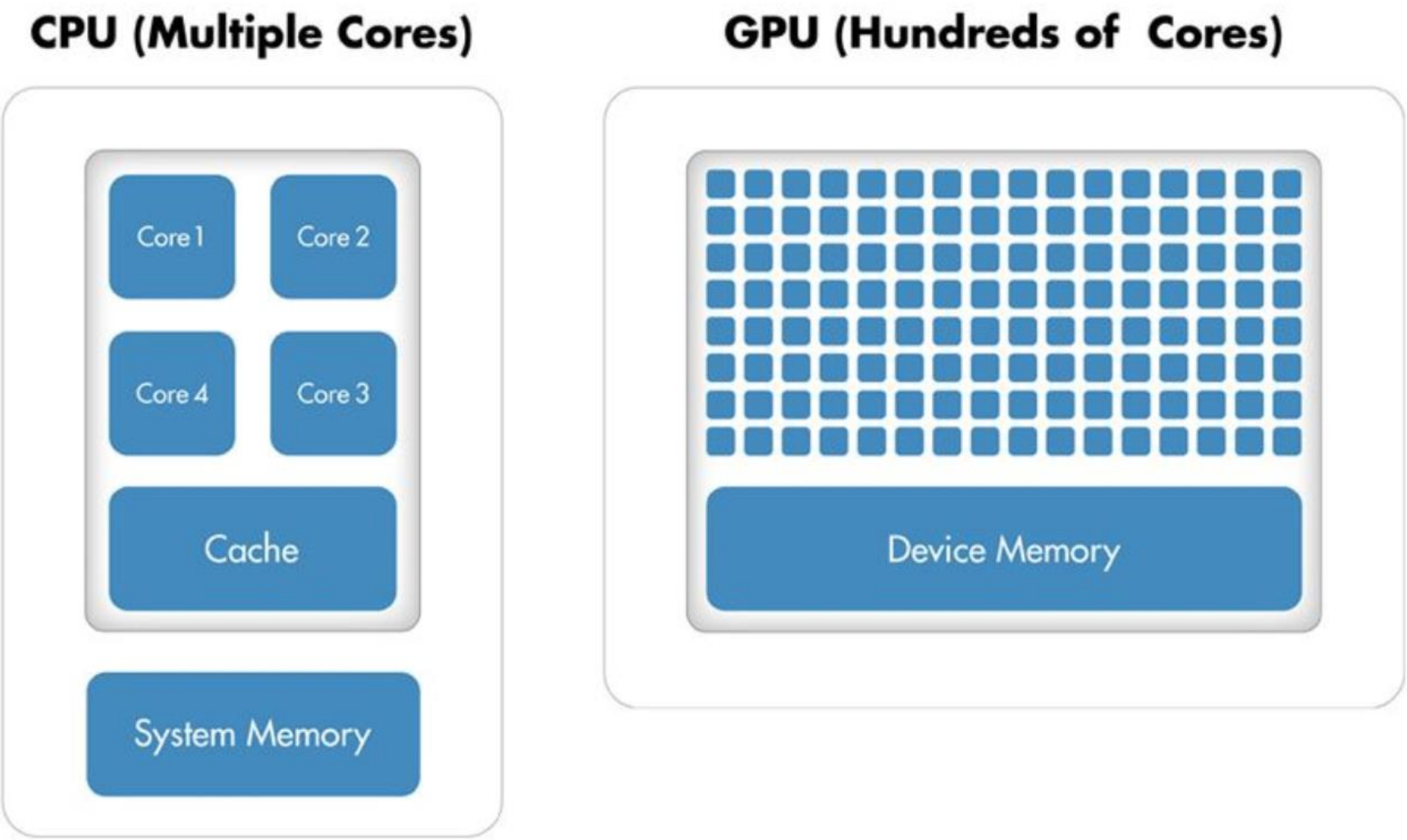
QUESTION: why **PyTorch + tensors** instead of **NumPy + numpy array** ?

ANSWER: 1) **GPU** is well-supported to accelerate the computation with ***tensor*** class, whereas ***numpy*** only supports **CPU** computation

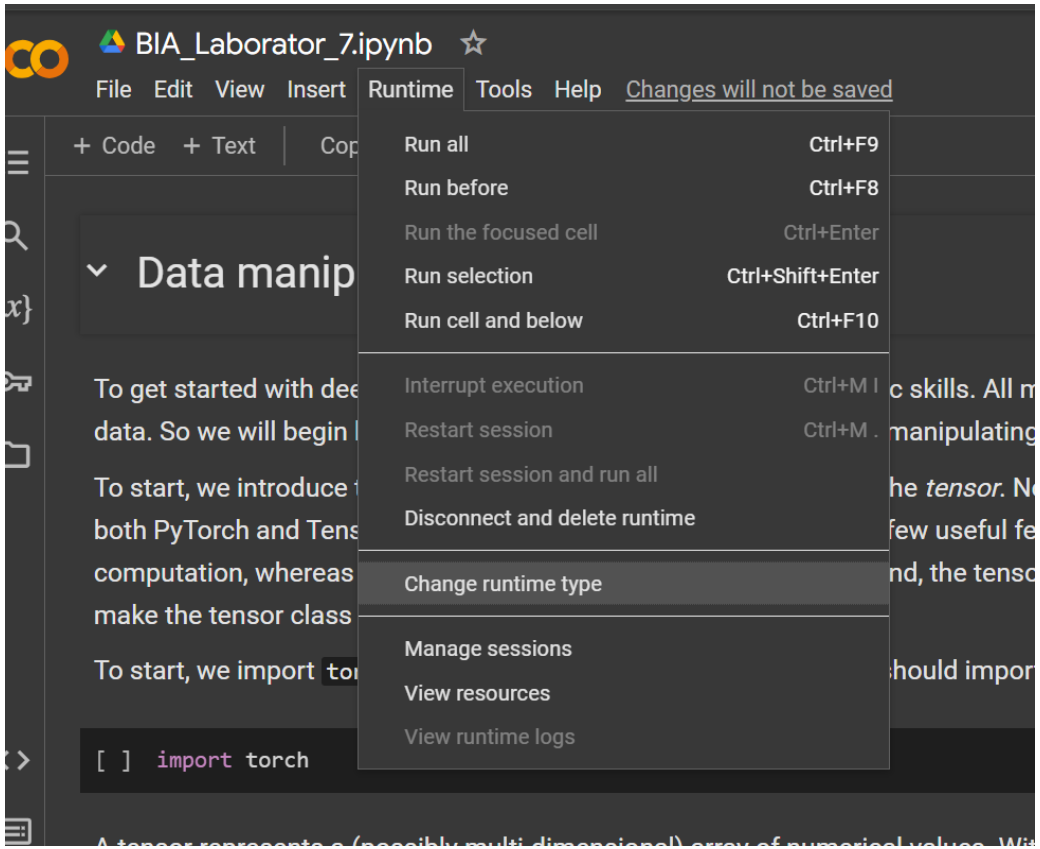
2) the tensor class supports **automatic differentiation**

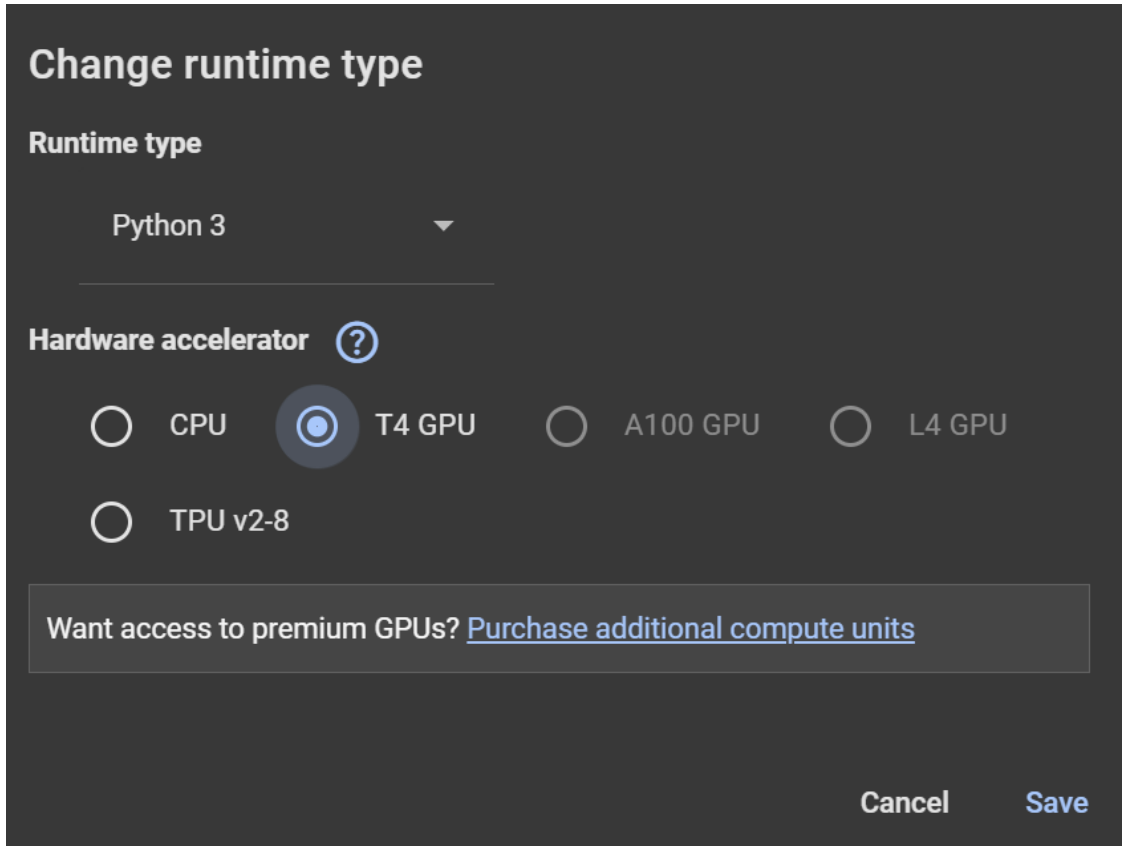


NOTE: In deep learning we have millions/billions of matrix multiplication operations !



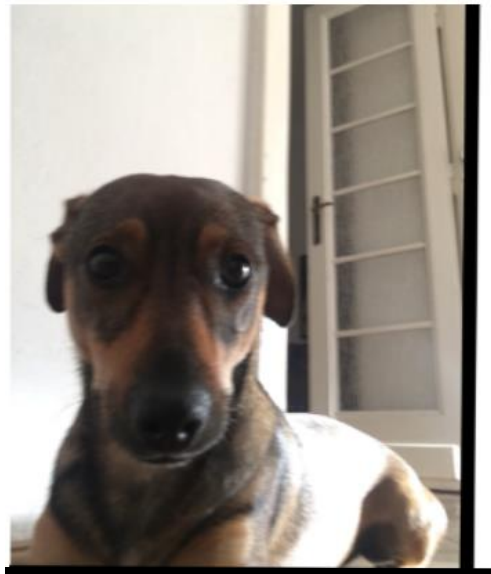
NOTE: neural networks involve operations that can be parallelized, a strength of GPUs


BIA_Laborator_7.ipynb
File Edit View Insert Runtime Tools Help Changes will not be saved
+ Code + Text
Run all Ctrl+F9
Run before Ctrl+F8
Run the focused cell Ctrl+Enter
Run selection Ctrl+Shift+Enter
Run cell and below Ctrl+F10
Interrupt execution Ctrl+M I
Restart session Ctrl+M .
Restart session and run all
Disconnect and delete runtime
Change runtime type
Manage sessions
View resources
View runtime logs
Data manip
[ ] import torch


Change runtime type
Runtime type
Python 3
Hardware accelerator
CPU
T4 GPU
A100 GPU
L4 GPU
TPU v2-8
Want access to premium GPUs? Purchase additional compute units
Cancel
Save

Automatic differentiation

- is a computational technique that automatically calculates the derivative of functions with respect to their inputs

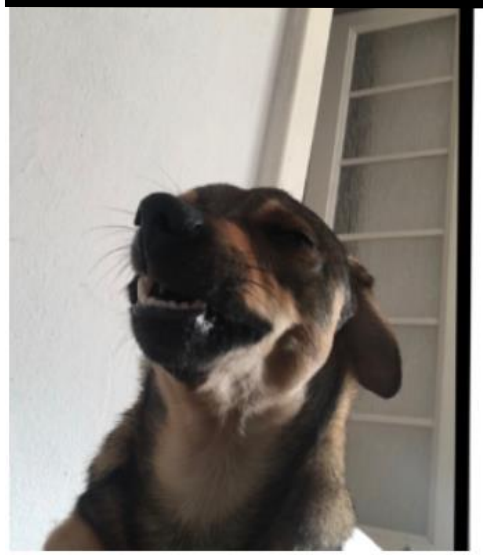


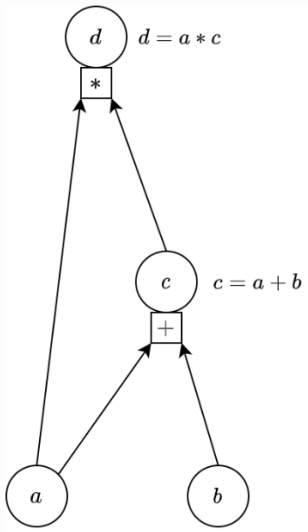
$$
\begin{aligned}
\frac{\partial L}{\partial \boldsymbol{h}} &= \operatorname{prod}\left(\frac{\partial L}{\partial \boldsymbol{o}}, \frac{\partial \boldsymbol{o}}{\partial \boldsymbol{h}}\right) \\
&= \operatorname{prod}\left(\frac{\partial L}{\partial \boldsymbol{o}}, \frac{\partial (\boldsymbol{W}^{(2)}\boldsymbol{h})}{\partial \boldsymbol{h}}\right) \\
&= \boldsymbol{W}^{(2)\top}\frac{\partial L}{\partial \boldsymbol{o}} \\
&= \boldsymbol{W}^{(2)\top}(\boldsymbol{o}-\boldsymbol{y}).
\end{aligned}
$$

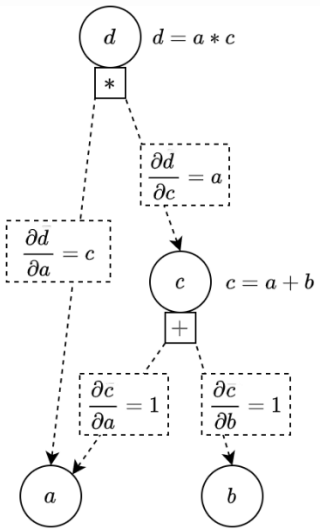
$$\frac{\partial L}{\partial \boldsymbol{z}} = \left(\boldsymbol{W}^{(2)\top}(\boldsymbol{o}-\boldsymbol{y})\right) \odot \phi'(\boldsymbol{z}).$$

$$\frac{\partial s}{\partial \boldsymbol{W}^{(1)}} = \lambda \boldsymbol{W}^{(1)}.$$

$$\frac{\partial J}{\partial \boldsymbol{W}^{(1)}} = \left(\left(\boldsymbol{W}^{(2)\top}(\boldsymbol{o}-\boldsymbol{y})\right) \odot \phi'(\boldsymbol{z})\right)\boldsymbol{x}^\top + \lambda \boldsymbol{W}^{(1)}.$$







we will explain this soon ☺

```python
import torch
```

NOTE: although it's called PyTorch, we should import **torch**, not **pytorch**

```python
x = torch.arange(12, dtype=torch.float32)
x
```

```
tensor([ 0.,  1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9., 10., 11.])
```

```python
x.shape
```

```
torch.Size([12])
```

```python
X = x.reshape(3, 4)
X
```

```
tensor([[ 0.,  1.,  2.,  3.],
        [ 4.,  5.,  6.,  7.],
        [ 8.,  9., 10., 11.]])
```

!

NOTE: Unless otherwise specified, new tensors are stored in main memory and designated for CPU-based computation !

```python
x.device
```

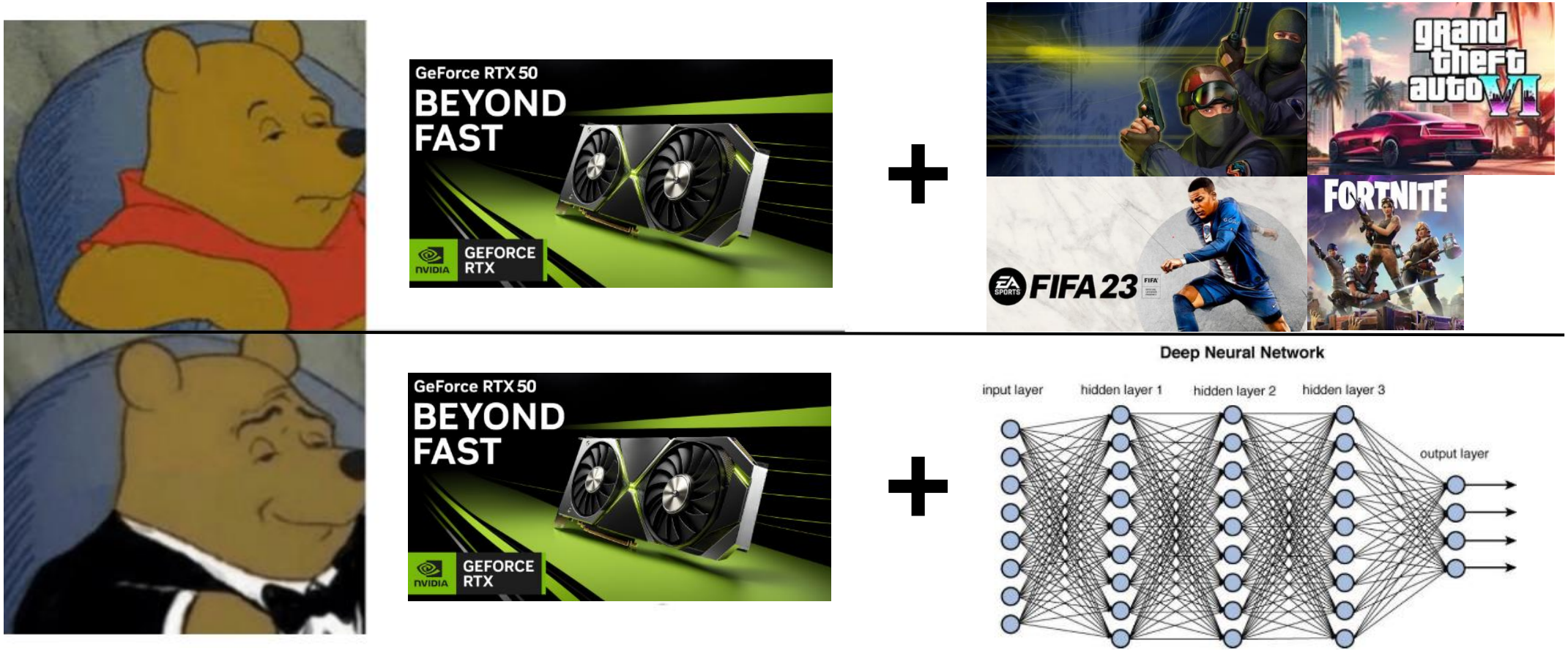
```
device(type='cpu')
```

```python
x = x.to('cuda')
```

```python
x.device
```

```
device(type='cuda', index=0)
```

- CUDA is a special technology from NVIDIA that allows its GPUs to do more than just graphics.
- normally, GPUs are used to render images or videos, but CUDA turns them into powerful calculators.
- this is helpful for tasks like training AI models, where we need to do a huge number of calculations very quickly.



Many ways to create tensors...

```
torch.zeros((2, 3, 4))
```

```
tensor([[[0., 0., 0., 0.],
         [0., 0., 0., 0.],
         [0., 0., 0., 0.]],

        [[0., 0., 0., 0.],
         [0., 0., 0., 0.],
         [0., 0., 0., 0.]]])
```

```
torch.ones((2, 3, 4))
```

```
tensor([[[1., 1., 1., 1.],
         [1., 1., 1., 1.],
         [1., 1., 1., 1.]],

        [[1., 1., 1., 1.],
         [1., 1., 1., 1.],
         [1., 1., 1., 1.]]])
```

```
torch.randn(3, 4)
```

```
tensor([[ 0.5189,  1.1389, -0.9604, -2.4524],
        [ 0.2156,  0.2228,  1.8153,  0.6495],
        [-1.8379, -0.6394,  0.5575, -0.8809]])
```

```
torch.tensor([[2, 1, 4, 3], [1, 2, 3, 4], [4, 3, 2, 1]])
```

```
tensor([[2, 1, 4, 3],
        [1, 2, 3, 4],
        [4, 3, 2, 1]])
```

Many mathematical operations...

```
x = torch.tensor([1.0, 2, 4, 8])
y = torch.tensor([2, 2, 2, 2])
x + y, x - y, x * y, x / y, x ** y  # The ** operator is exponentiation
```

```
(tensor([ 3.,  4.,  6., 10.]),
 tensor([-1.,  0.,  2.,  6.]),
 tensor([ 2.,  4.,  8., 16.]),
 tensor([0.5000, 1.0000, 2.0000, 4.0000]),
 tensor([ 1.,  4., 16., 64.]))
```

```
torch.exp(x)
```

```
tensor([2.7183e+00, 7.3891e+00, 5.4598e+01, 2.9810e+03])
```

Ok, maybe not so many here 😀

Other useful stuffs...

- converting from *numpy array* to *tensor* (or viceversa):

```
X = torch.tensor([[2, 1, 4, 3], [1, 2, 3, 4], [4, 3, 2, 1]])
X_numpy = X.numpy()
X_tensor = torch.from_numpy(X_numpy)
type(X_numpy), type(X_tensor)
```

- concatenating 2 tensors on different axis:

```
tensor_a = torch.tensor([[1, 2], [3, 4]])
tensor_b = torch.tensor([[5, 6], [7, 8]])

print(f"Tensor A ({tensor_a.shape}): \n", tensor_a)
print(f"Tensor B ({tensor_b.shape}): \n", tensor_b)
```

```
Tensor A (torch.Size([2, 2])):
 tensor([[1, 2],
        [3, 4]])
Tensor B (torch.Size([2, 2])):
 tensor([[5, 6],
        [7, 8]])
```

```
concat_axis_0 = torch.cat((tensor_a, tensor_b), dim=0)
print(f"Concatenated on Axis 0 ({concat_axis_0.shape}): \n", concat_axis_0
```

```
Concatenated on Axis 0 (torch.Size([4, 2])):
 tensor([[1, 2],
        [3, 4],
        [5, 6],
        [7, 8]])
```

```
concat_axis_1 = torch.cat((tensor_a, tensor_b), dim=1)
print(f"Concatenated on Axis 1 ({concat_axis_1.shape}): \n", concat_axis_1)
```

```
Concatenated on Axis 1 (torch.Size([2, 4])):
 tensor([[1, 2, 5, 6],
        [3, 4, 7, 8]])
```

Otherwise we have the same operations as with numpy arrays...

```
X = torch.arange(12, dtype=torch.float32).reshape((3,4))
X
```

```
tensor([[ 0.,  1.,  2.,  3.],
        [ 4.,  5.,  6.,  7.],
        [ 8.,  9., 10., 11.]])
```

select the last element

```
X[-1], X[1:3]
```

```
(tensor([ 8.,  9., 10., 11.]), tensor([[ 4.,  5.,  6.,  7.],
         [ 8.,  9., 10., 11.]]))
```

```
X[1, 2] = 9
X
```

```
tensor([[ 0.,  1.,  2.,  3.],
        [ 4.,  5.,  9.,  7.],
        [ 8.,  9., 10., 11.]])
```

```
X[0:2, :] = 12
X
```

```
tensor([[12., 12., 12., 12.],
        [12., 12., 12., 12.],
        [ 8.,  9., 10., 11.]])
```

2) Linear algebra in PyTorch

```python
A = torch.arange(20).reshape(5, 4)
A
```

```
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11],
        [12, 13, 14, 15],
        [16, 17, 18, 19]])
```

```python
A.T
```

```
tensor([[ 0,  4,  8, 12, 16],
        [ 1,  5,  9, 13, 17],
        [ 2,  6, 10, 14, 18],
        [ 3,  7, 11, 15, 19]])
```

```python
B = A.clone()  # Assign a copy of `A` to `B` by allocating new memory
A * B
```

```
tensor([[  0.,   1.,   4.,   9.],
        [ 16.,  25.,  36.,  49.],
        [ 64.,  81., 100., 121.],
        [144., 169., 196., 225.],
        [256., 289., 324., 361.]])
```

elementwise multiplication

```python
A_sum_axis0 = A.sum(axis=0)
A_sum_axis0
```

```
(tensor([40., 45., 50., 55.])
```

```python
A_sum_axis1 = A.sum(axis=1)
A_sum_axis1
```

```
(tensor([ 6., 22., 38., 54., 70.])
```

```python
A.sum()
```

```
tensor(190.))
```

Matrix-matrix multiplication

```
A = torch.arange(20).reshape(5, 4)
A
```

```
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11],
        [12, 13, 14, 15],
        [16, 17, 18, 19]])
```

```
B = torch.ones(4, 3)
B
```

```
tensor([[1., 1., 1.],
        [1., 1., 1.],
        [1., 1., 1.],
        [1., 1., 1.]])
```

```
A @ B
```

```
tensor([[ 6.,  6.,  6.],
        [22., 22., 22.],
        [38., 38., 38.],
        [54., 54., 54.],
        [70., 70., 70.]])
```

```
torch.mm(A, B)
```

```
tensor([[ 6.,  6.,  6.],
        [22., 22., 22.],
        [38., 38., 38.],
        [54., 54., 54.],
        [70., 70., 70.]])
```

```python
u = torch.tensor([3.0, -4.0])
torch.norm(u)
```

```
tensor(5.)
```

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^{n} x_i^2}$$

l_1 norm

```python
torch.abs(u).sum()
```

```
tensor(7.)
```

$$\|\mathbf{x}\|_1 = \sum_{i=1}^{n} |x_i|$$

l_2 norm

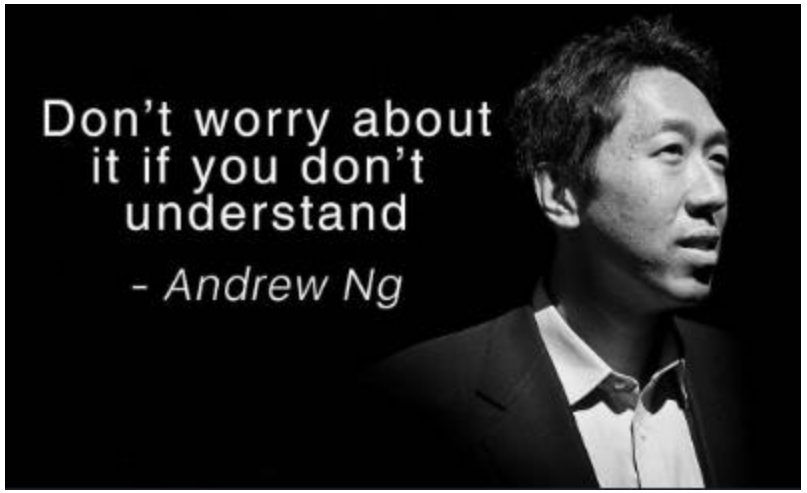
```python
torch.norm(torch.ones((4, 9)))
```

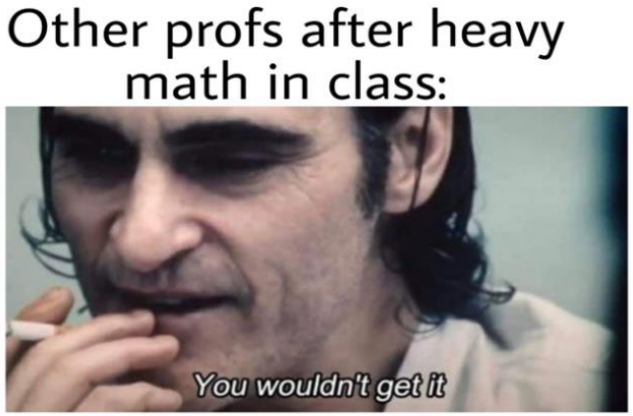
```
tensor(6.)
```

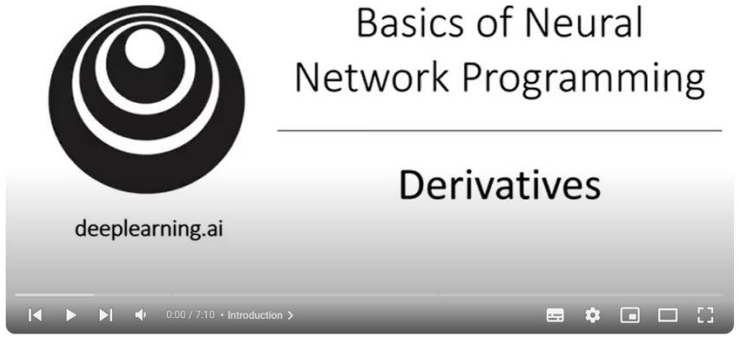
$$\|\mathbf{X}\|_F = \sqrt{\sum_{i=1}^{m} \sum_{j=1}^{n} x_{ij}^2}$$

Frobenius norm

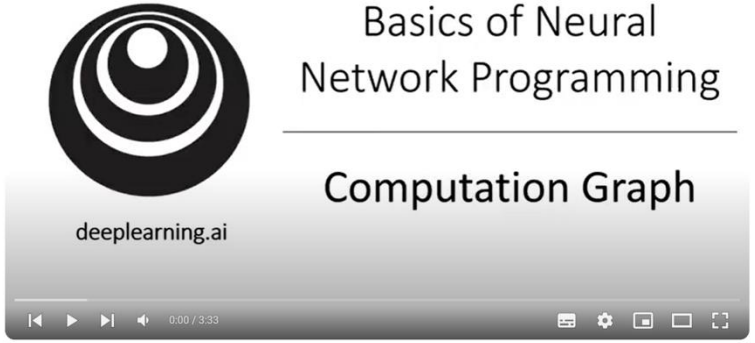
3) Automatic differentiation in PyTorch



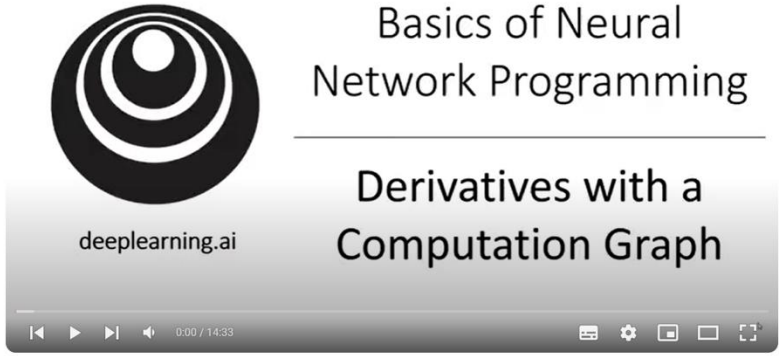




Derivatives (C1W2L05)

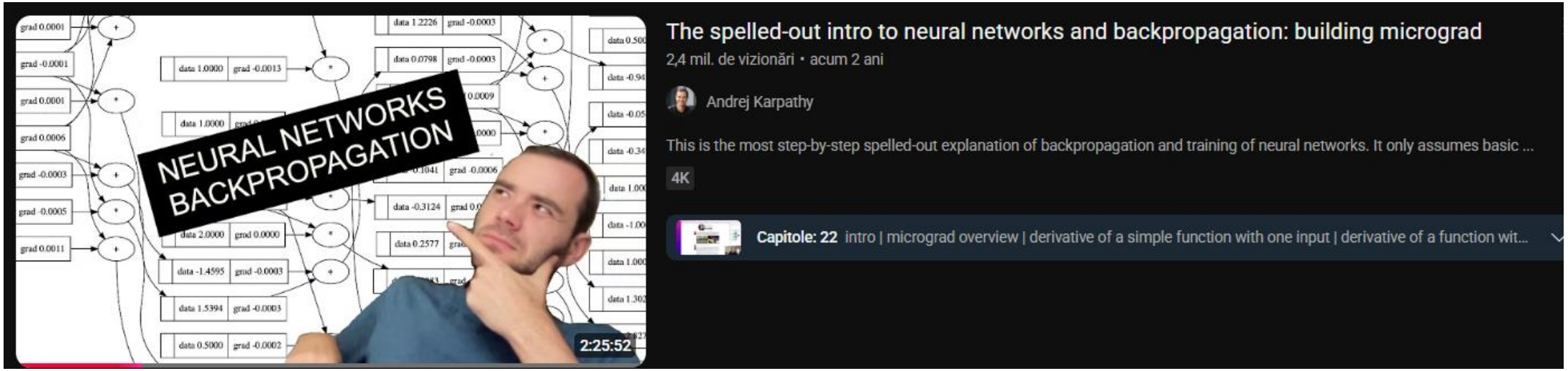


Computation Graph (C1W2L07)



Derivatives With Computation Graphs (C1W2L08)

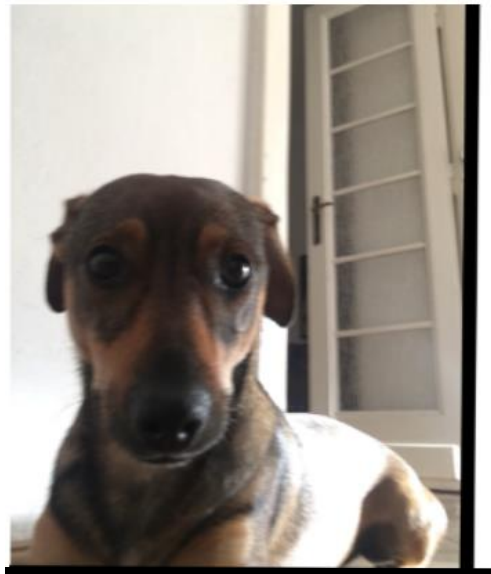
...one of the best video explaining backpropagation:



Andrej Karpathy – co-founded OpenAI

Automatic differentiation (back here ☺)

- is a computational technique that automatically calculates the derivative of functions with respect to their inputs

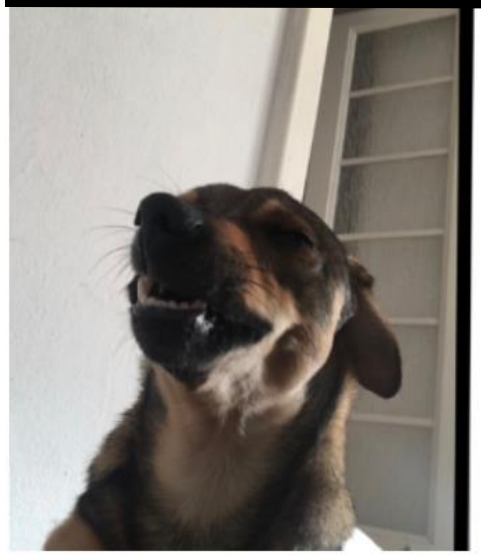


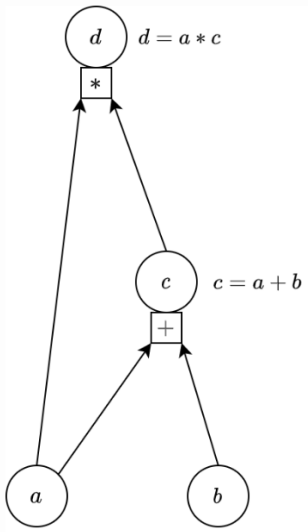
$$
\begin{aligned}
\frac{\partial L}{\partial \boldsymbol{h}} &= \operatorname{prod}\left(\frac{\partial L}{\partial \boldsymbol{o}}, \frac{\partial \boldsymbol{o}}{\partial \boldsymbol{h}}\right) \\
&= \operatorname{prod}\left(\frac{\partial L}{\partial \boldsymbol{o}}, \frac{\partial (\boldsymbol{W}^{(2)}\boldsymbol{h})}{\partial \boldsymbol{h}}\right) \\
&= \boldsymbol{W}^{(2)\top}\frac{\partial L}{\partial \boldsymbol{o}} \\
&= \boldsymbol{W}^{(2)\top}(\boldsymbol{o}-\boldsymbol{y}).
\end{aligned}
$$

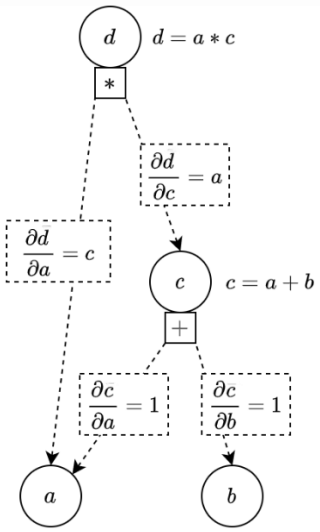
$$
\frac{\partial L}{\partial \boldsymbol{z}} = \left(\boldsymbol{W}^{(2)\top}(\boldsymbol{o}-\boldsymbol{y})\right) \odot \phi'(\boldsymbol{z}).
$$

$$
\frac{\partial s}{\partial \boldsymbol{W}^{(1)}} = \lambda \boldsymbol{W}^{(1)}.
$$

$$
\frac{\partial J}{\partial \boldsymbol{W}^{(1)}} = \left(\left(\boldsymbol{W}^{(2)\top}(\boldsymbol{o}-\boldsymbol{y})\right) \odot \phi'(\boldsymbol{z})\right)\boldsymbol{x}^{\top} + \lambda \boldsymbol{W}^{(1)}.
$$







- differentiation is a crucial step in nearly all deep learning problems

  = calcularea de derivate/gradienți

- while the calculations for taking these derivatives are straightforward, requiring only some basic calculus, for complex models, working out the updates by hand can be very complicated (and often error-prone)

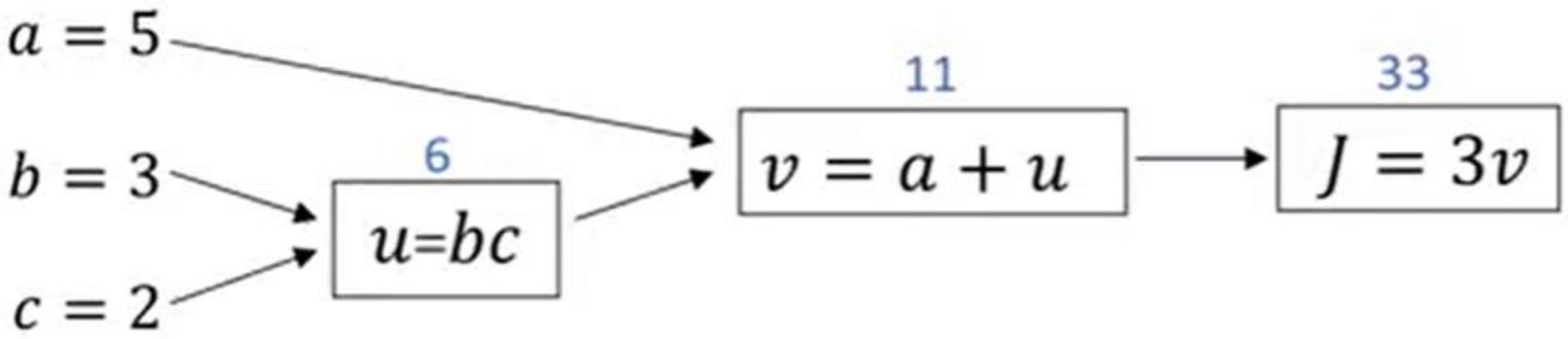
- deep learning frameworks (PyTorch, Keras, TensorFlow) speed up this work by automatically calculating derivatives, i.e., *automatic differentiation*

- in practice, based on our designed model/neural network, the system builds a computational graph, tracking which data combined through which operations to produce the output

- *automatic differentiation* enables the system to subsequently backpropagate gradients. Here, backpropagate simply means to trace through the computational graph, filling in the partial derivatives with respect to each parameter

A simple example

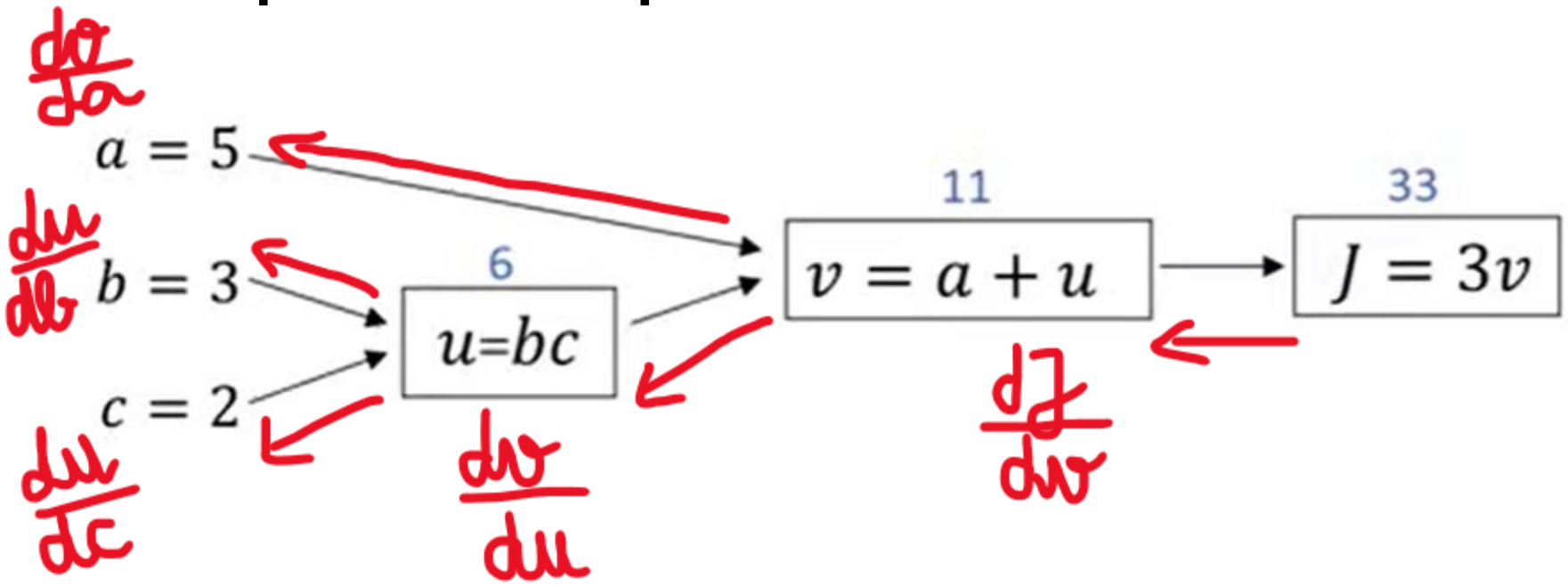
Our goal: find $\frac{dJ}{da}$, $\frac{dJ}{db}$, $\frac{dJ}{dc}$



$$\frac{dJ}{dv} = 3 \qquad \frac{du}{db} = c = 2$$

$$\frac{dv}{du} = 1 \qquad \frac{dv}{da} = 1 \qquad \frac{du}{dc} = b = 3$$

A simple example



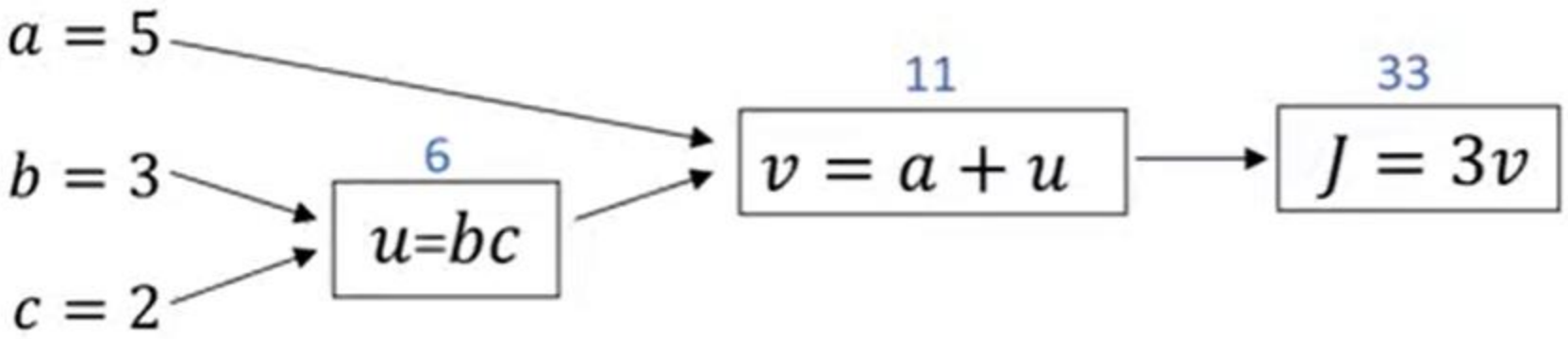
Now we use **chain rule** from high school: $F'(x) = f'(g(x)) \cdot g'(x)$ where $F(x) = f(g(x))$

or

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$ → **we will use this form !**

A simple example

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$



$$\frac{dJ}{da} = \frac{dJ}{dv} \cdot \frac{dv}{da} = 3 \cdot 1 = 3$$

$$\frac{dJ}{db} = \frac{dJ}{dv} \cdot \frac{dv}{du} \cdot \frac{du}{db} = 3 \cdot 1 \cdot 2 = 6$$

$$\frac{dJ}{dc} = \frac{dJ}{dv} \cdot \frac{dv}{du} \cdot \frac{du}{dc} = 3 \cdot 1 \cdot 3 = 9$$

```
x = torch.arange(4.0)
x
```

```
tensor([0., 1., 2., 3.])
```

this tells PyTorch to track operations on the tensor and build a **computational graph** for automatic differentiation (the graph from the previous slides)

or we can simply do

```
x = torch.arange(4.0, requires_grad=True)
```

```
x.requires_grad_(True)
```

```
print(x.grad)
```

```
None
```

```
y = 2 * torch.dot(x, x)
y
```

```
tensor(28., grad_fn=<MulBackward0>)
```

```
y.backward()
x.grad
```

```
tensor([ 0.,  4.,  8., 12.])
```

$$x = [x_1, x_2, x_3, x_4] = [0, 1, 2, 3]$$

$$y = 2 \cdot \text{torch.dot}(x, x)$$

$$\text{torch.dot}(x, x) = x_1^2 + x_2^2 + x_3^2 + x_4^2$$

Substituting $x = [0, 1, 2, 3]$:

$$\text{torch.dot}(x, x) = 0^2 + 1^2 + 2^2 + 3^2 = 0 + 1 + 4 + 9 = 14$$

$$\Rightarrow \quad y = 2 \cdot 14 = 28$$

Since $y = 2 \cdot (x_1^2 + x_2^2 + x_3^2 + x_4^2)$, we can find the partial derivatives with respect to each x_i:

$$\frac{\partial y}{\partial x_i} = 2 \cdot \frac{\partial}{\partial x_i}(x_1^2 + x_2^2 + x_3^2 + x_4^2)$$

Because only x_i affects x_i^2, we get:

$$\frac{\partial y}{\partial x_i} = 2 \cdot 2x_i = 4x_i$$

Using the formula $\frac{\partial y}{\partial x_i} = 4x_i$:

- For $x_1 = 0$: $\frac{\partial y}{\partial x_1} = 4 \cdot 0 = 0$
- For $x_2 = 1$: $\frac{\partial y}{\partial x_2} = 4 \cdot 1 = 4$
- For $x_3 = 2$: $\frac{\partial y}{\partial x_3} = 4 \cdot 2 = 8$
- For $x_4 = 3$: $\frac{\partial y}{\partial x_4} = 4 \cdot 3 = 12$

Other example

```
x.grad.zero_()
y = x.sum()
```

```
y.backward()
x.grad
```

```
tensor([1., 1., 1., 1.])
```

NOTE:

- this operation is essential to avoid accumulating gradients from previous operations
- in PyTorch, the gradients are accumulated (summed up) by default in the *x.grad* attribute whenever *backward()* is called
- this is helpful for cases where you want to accumulate gradients across multiple operations, but it can lead to incorrect or unintended results if you're not explicitly aiming for that

$$y = x.\text{sum}()$$

which computes the sum of all elements in x:

$$y = x_1 + x_2 + x_3 + x_4 = 0 + 1 + 2 + 3 = 6$$

Since $y = x_1 + x_2 + x_3 + x_4$, let's find the partial derivative of y with respect to each x_i:

$$\frac{\partial y}{\partial x_i} = 1$$

This is because each x_i contributes linearly to the sum, so the derivative of x_i with respect to itself is simply 1.

After calling `y.backward()`, each element in $x.grad$ will be set to 1, because $\frac{\partial y}{\partial x_i} = 1$ for each i:

$$x.grad = [1., 1., 1., 1.]$$, q.e.d.

Exercises

Laboratory 7 – Exercises

1. Create tensors of the following ranks: 1, 2, and 3. Use values between 0 and 1 to fill your tensors. The size of the tensors can be defined as you wish, given that the ranks are created correctly. Print the shape of each of the tensors using the `shape` property.

2. The **sigmoid** activation function is an essential function in neural networks and is defined as follows:

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}.$$

Take the derivative of this function with respect to x, where $x = [-10, 10, 8]$.

3. Define the following two matrices:

$$A = \begin{bmatrix} 1 & 2 & -3 \\ 4 & 5 & 10 \end{bmatrix}, \quad B = \begin{bmatrix} 10 & 11 & -2 \\ 13 & -3 & 8 \end{bmatrix}.$$

- calculate the Hadamard product of the two matrices;
- add the scalar $a = 10$ to each element of matrix B;
- print the transpose of both matrices.

4. Create the following two matrices:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \quad B = \begin{bmatrix} 10 & 11 \\ -2 & 0 \\ 1 & -3 \end{bmatrix}.$$

Multiply the two matrices and compute the Frobenius norm of A and B.

Solutions

1. Create tensors of the following ranks: 1, 2, and 3. Use values between 0 and 1 to fill your tensors. The size of the tensors can be defined as you wish, given that the ranks are created correctly. Print the shape of each of the tensors using the **shape** property.

```python
import torch

# Rank 1 tensor (vector)
tensor_1 = torch.rand(5)  # 1D tensor with 5 elements
print("Rank 1 tensor shape:", tensor_1.shape)

# Rank 2 tensor (matrix)
tensor_2 = torch.rand(3, 4)  # 2D tensor with shape 3x4
print("Rank 2 tensor shape:", tensor_2.shape)

# Rank 3 tensor
tensor_3 = torch.rand(2, 3, 4)  # 3D tensor with shape 2x3x4
print("Rank 3 tensor shape:", tensor_3.shape)
```

```
Rank 1 tensor shape: torch.Size([5])
Rank 2 tensor shape: torch.Size([3, 4])
Rank 3 tensor shape: torch.Size([2, 3, 4])
```

2. The **sigmoid** activation function is an essential function in neural networks and is defined as follows:

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}.$$

Take the derivative of this function with respect to x, where $x = [-10, 10, 8]$.

```python
# Define the sigmoid function
def sigmoid(x):
    return 1 / (1 + torch.exp(-x))

# Create x as specified: x = [-10, 10, 8]
x = torch.tensor([-10.0, 10.0, 8.0], requires_grad=True)
y = sigmoid(x)

# Take the derivative of sigmoid
y.backward(torch.ones_like(x))  # Computes gradients
print("Derivative of sigmoid at x =", x.tolist(), "is", x.grad.tolist())
```

why this? ☺

```
Derivative of sigmoid at x = [-10.0, 10.0, 8.0] is [4.5395812776405364e-05, 4.539580550044775e-05, 0.0003352376807015389]
```

- by default, *y.backward()* computes the gradient of *y* with respect to *x*, assuming *y* is a scalar

- **however, if *y* is not a scalar** (e.g., *y* has multiple elements like in this case, where *y* has 3 elements), **PyTorch requires you to specify the gradient of *y* itself**

- this gradient is often called the "**vector of ones**"

- using ***torch.ones_like(x)*** effectively ensures you're computing the derivative of y with respect to *x* for each component of *y*, treating each output dimension equally (as having a gradient of 1). This is crucial for multi-dimensional tensors

- or you can use *y.sum()* to achieve the same effect, both variants are correct:

```python
# Define the sigmoid function
def sigmoid(x):
    return 1 / (1 + torch.exp(-x))

# Create x as specified: x = [-10, 10, 8]
x = torch.tensor([-10.0, 10.0, 8.0], requires_grad=True)
y = sigmoid(x)

# Take the derivative of sigmoid
y.sum().backward()
print("Derivative of sigmoid at x =", x.tolist(), "is", x.grad.tolist())
```

```
Derivative of sigmoid at x = [-10.0, 10.0, 8.0] is [4.5395812776405364e-05, 4.539580550044775e-05, 0.0003352376807015389]
```

- why .sum() works?
  - *y.sum()* collapses the tensor *y* into a scalar by summing all its elements
  - during backpropagation, the gradient of the sum operation assigns a gradient of 1 to each element of *y* (since the derivative of a sum with respect to each term is 1)
  - this is effectively the same as providing *torch.ones_like(x)* in *backward()*

3. Define the following two matrices:

$$A = \begin{bmatrix} 1 & 2 & -3 \\ 4 & 5 & 10 \end{bmatrix}, \quad B = \begin{bmatrix} 10 & 11 & -2 \\ 13 & -3 & 8 \end{bmatrix}.$$

- calculate the Hadamard product of the two matrices;
- add the scalar $a = 10$ to each element of matrix B;
- print the transpose of both matrices.

```python
A = torch.tensor([[1, 2, -3], [4, 5, 10]], dtype=torch.float32)
B = torch.tensor([[10, 11, -2], [13, -3, 8]], dtype=torch.float32)

# Hadamard (element-wise) product
hadamard_product = A * B
print("Hadamard product of A and B:\n", hadamard_product)

# Add scalar a = 10 to each element of matrix B
a = 10
B_added = B + a
print("Matrix B after adding scalar 10:\n", B_added)

# Print transposes of both matrices
print("Transpose of matrix A:\n", A.T)
print("Transpose of matrix B:\n", B.T)
```

```
Hadamard product of A and B:
 tensor([[ 10.,  22.,   6.],
        [ 52., -15.,  80.]])
Matrix B after adding scalar 10:
 tensor([[20., 21.,  8.],
        [23.,  7., 18.]])
Transpose of matrix A:
 tensor([[ 1.,  4.],
        [ 2.,  5.],
        [-3., 10.]])
Transpose of matrix B:
 tensor([[10., 13.],
        [11., -3.],
        [-2.,  8.]])
```

4. Create the following two matrices:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \quad B = \begin{bmatrix} 10 & 11 \\ -2 & 0 \\ 1 & -3 \end{bmatrix}.$$

Multiply the two matrices and compute the Frobenius norm of A and B.

```python
A = torch.tensor([[1, 2, 3], [4, 5, 6]], dtype=torch.float32)
B = torch.tensor([[10, 11, 2], [-2, 0, 1], [1, -3, 0]], dtype=torch.float32)

# Matrix multiplication of A2 and B2
product = torch.mm(A, B)
print("Matrix multiplication of A and B:\n", product)

# Compute the Frobenius norm of A and B
frobenius_norm_A = torch.norm(A, p='fro')
frobenius_norm_B = torch.norm(B, p='fro')
print("Frobenius norm of A:", frobenius_norm_A.item())
print("Frobenius norm of B:", frobenius_norm_B.item())
```

```
Matrix multiplication of A and B:
 tensor([[ 9.,  2.,  4.],
        [36., 26., 13.]])
Frobenius norm of A: 9.539392471313477
Frobenius norm of B: 15.491933822631836
```

Thank you !